

Name: CH. Durga Prasad

Reg.no:192425305

Course Code: CSA0603

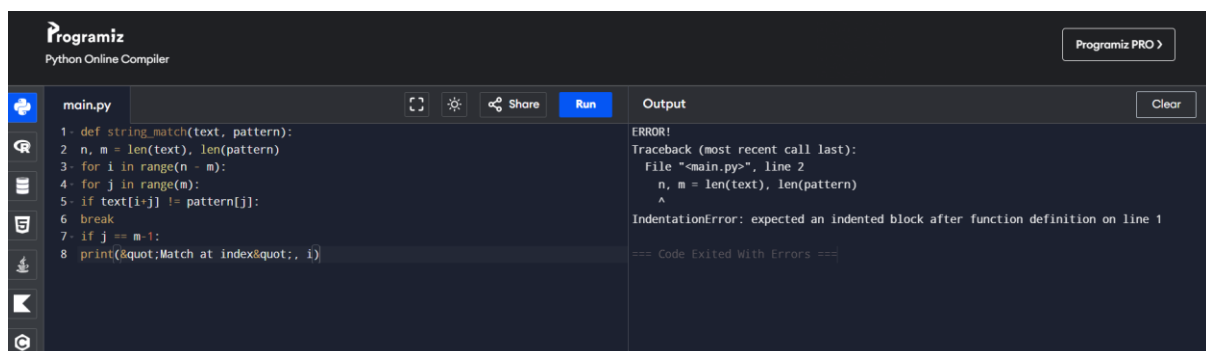
Q8. Debugging String Matching Code

The given brute-force string matching code fails to detect all pattern occurrences due to incorrect loop bounds. Identify logical errors in indexing and comparison. Explain why some matches are skipped. Debug the code to correctly find all occurrences. Optimize the implementation to avoid redundant comparisons. Analyze the time complexity of the corrected version. Rewrite the final code with proper comments.

Original code:

```
def string_match(text, pattern):  
  
    n, m = len(text), len(pattern)  
  
    for i in range(n - m):  
  
        for j in range(m):  
  
            if text[i+j] != pattern[j]:  
  
                break  
  
            if j == m-1:  
  
                print("&quot;Match at index&quot;, i)
```

Output:



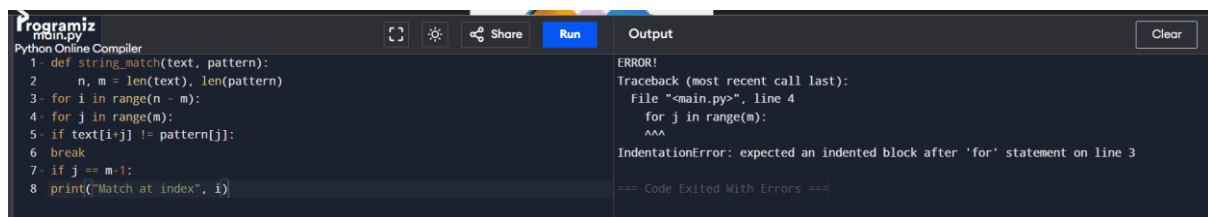
The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the original code with several syntax errors. The output panel on the right displays a traceback error: "IndentationError: expected an indented block after function definition on line 1". The error message is shown in a dark-themed window with a "Clear" button.

The code has six mistakes: missing indentation, incorrect outer loop range (`range(n - m)` should be `range(n - m + 1)`), invalid HTML entity (`"` instead of `"`), and incorrect placement/indentation of the `if j == m - 1` condition after the inner loop.

1st correction: Missing indentation

```
def string_match(text, pattern):  
    n, m = len(text), len(pattern)  
  
    for i in range(n - m):  
  
        for j in range(m):  
  
            if text[i+j] != pattern[j]:  
  
                break  
  
        if j == m-1:  
  
            print("Match at index", i)
```

Output:



The screenshot shows a Python Online Compiler interface. The code editor on the left contains the following code:

```
1- def string_match(text, pattern):  
2-     n, m = len(text), len(pattern)  
3-     for i in range(n - m):  
4-     for j in range(m):  
5-     if text[i+j] != pattern[j]:  
6-     break  
7-     if j == m-1:  
8-     print("Match at index", i)
```

The output panel on the right shows an error message:

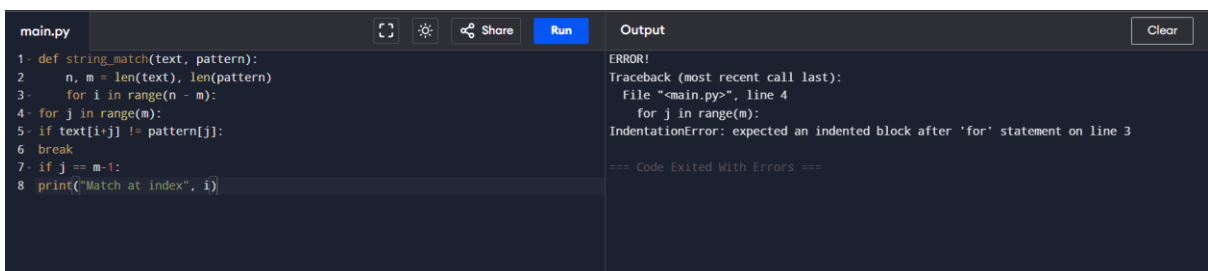
```
ERROR!  
Traceback (most recent call last):  
  File "<main.py>", line 4  
    for j in range(m):  
        ^^^  
IndentationError: expected an indented block after 'for' statement on line 3  
  
=== Code Exited With Errors ===
```

Only the **first mistake** (indentation inside the function) has been corrected. The remaining mistakes are left unchanged.

2nd mistake: Missing indentation inside the outer for loop

```
def string_match(text, pattern):  
    n, m = len(text), len(pattern)  
  
    for i in range(n - m):  
  
        for j in range(m):  
  
            if text[i+j] != pattern[j]:  
  
                break  
  
        if j == m-1:  
  
            print("Match at index", i)
```

Output:



The screenshot shows a Python Online Compiler interface. The code editor on the left contains the following code:

```
1- def string_match(text, pattern):  
2-     n, m = len(text), len(pattern)  
3-     for i in range(n - m):  
4-     for j in range(m):  
5-     if text[i+j] != pattern[j]:  
6-     break  
7-     if j == m-1:  
8-     print("Match at index", i)
```

The output panel on the right shows an error message:

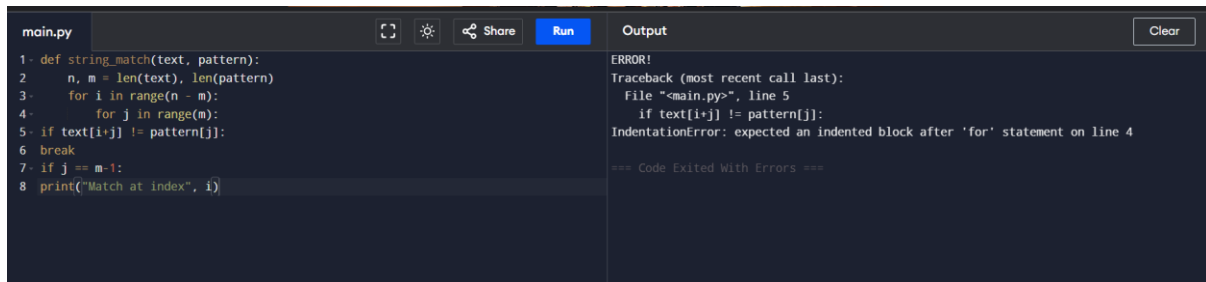
```
ERROR!  
Traceback (most recent call last):  
  File "<main.py>", line 4  
    for j in range(m):  
        ^^^  
IndentationError: expected an indented block after 'for' statement on line 3  
  
=== Code Exited With Errors ===
```

Only the second mistake (indentation of the outer for loop) has been corrected. The remaining mistakes are still unchanged.

3rd mistake: Missing indentation inside the inner for loop

```
def string_match(text, pattern):  
    n, m = len(text), len(pattern)  
    for i in range(n - m):  
        for j in range(m):  
            if text[i+j] != pattern[j]:  
                break  
        if j == m-1:  
            print("Match at index", i)
```

Output:



```
main.py  Run  Output  Clear  
1. def string_match(text, pattern):  
2.     n, m = len(text), len(pattern)  
3.     for i in range(n - m):  
4.         for j in range(m):  
5.             if text[i+j] != pattern[j]:  
6.                 break  
7.         if j == m-1:  
8.             print("Match at index", i)
```

ERROR!
Traceback (most recent call last):
 File "<main.py>", line 5
 if text[i+j] != pattern[j]:
IndentationError: expected an indented block after 'for' statement on line 4

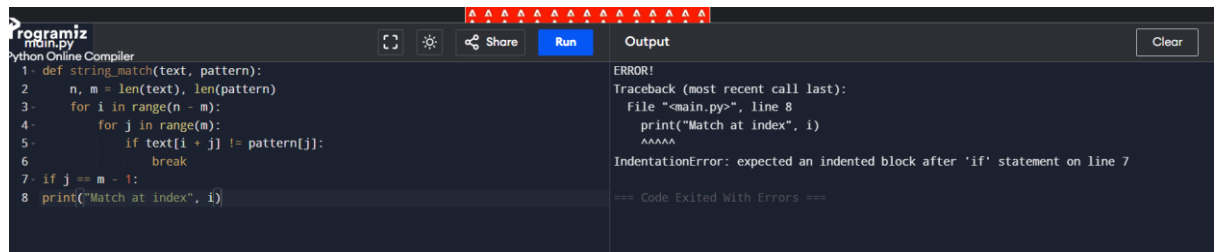
=== Code Exited With Errors ===

Only the third mistake (indentation of the inner for loop) has been corrected. The remaining mistakes are still unchanged.

4th mistake: Missing indentation inside the if statement

```
def string_match(text, pattern):  
    n, m = len(text), len(pattern)  
    for i in range(n - m):  
        for j in range(m):  
            if text[i + j] != pattern[j]:  
                break  
        if j == m - 1:  
            print("Match at index", i)
```

output:



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the following Python code:

```
1- def string_match(text, pattern):
2-     n, m = len(text), len(pattern)
3-     for i in range(n - m):
4-         for j in range(m):
5-             if text[i + j] != pattern[j]:
6-                 break
7-         if j == m - 1:
8-             print("Match at index", i)
```

The output panel on the right displays an error message:

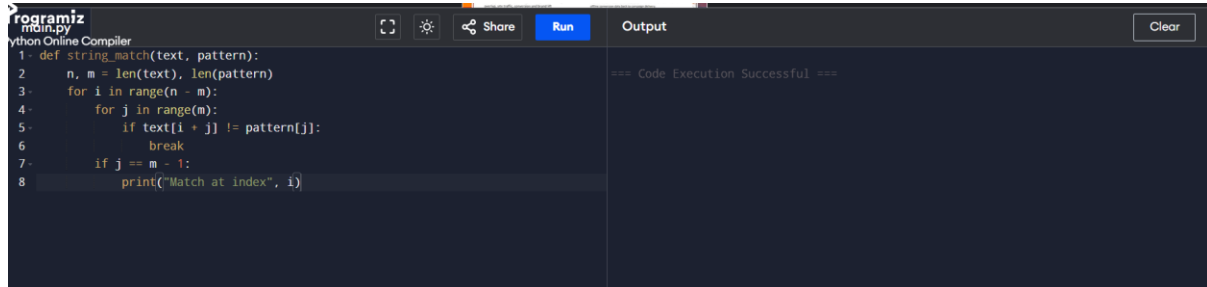
```
ERROR!
Traceback (most recent call last):
  File "<main.py>", line 8
    print("Match at index", i)
    ^^^^^
IndentationError: expected an indented block after 'if' statement on line 7

=== Code Exited With Errors ===
```

5th mistake: Missing indentation for the final if statement

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    for i in range(n - m):
        for j in range(m):
            if text[i + j] != pattern[j]:
                break
        if j == m - 1:
            print("Match at index", i)
```

Output:



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the same Python code as in the previous screenshot:

```
1- def string_match(text, pattern):
2-     n, m = len(text), len(pattern)
3-     for i in range(n - m):
4-         for j in range(m):
5-             if text[i + j] != pattern[j]:
6-                 break
7-         if j == m - 1:
8-             print("Match at index", i)
```

The output panel on the right displays the message:

```
=== Code Execution Successful ===
```

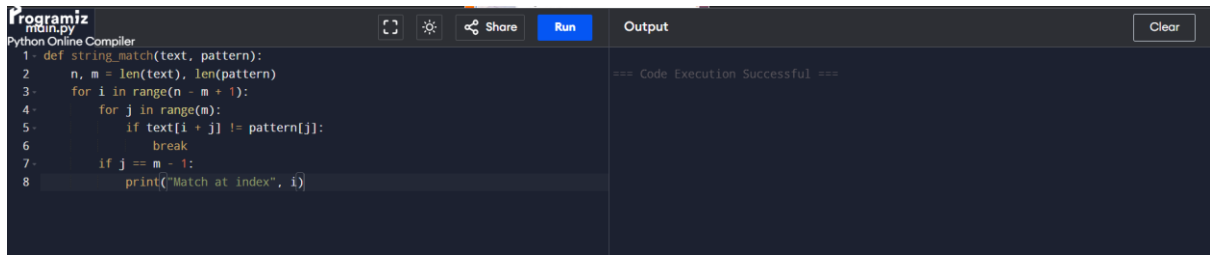
Only the fifth mistake (indentation of the final `if j == m - 1:` statement and its print statement) has been corrected. The 6th mistake (incorrect outer loop range `range(n - m)` should be `range(n - m + 1)`) is still unchanged.

6th mistake: Incorrect range in the outer loop

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    for i in range(n - m + 1):
        for j in range(m):
            if text[i + j] != pattern[j]:
                break
        if j == m - 1:
```

```
print("Match at index", i)
```

output:



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains the following Python code:

```
1- def string_match(text, pattern):
2-     n, m = len(text), len(pattern)
3-     for i in range(n - m + 1):
4-         for j in range(m):
5-             if text[i + j] != pattern[j]:
6-                 break
7-             if j == m - 1:
8-                 print("Match at index", i)
```

The output panel on the right displays the result of the code execution:

```
=== Code Execution Successful ===
```

Now all 6 mistakes have been corrected. (A more Pythonic solution uses a for...else instead of if j == m - 1, but the above fixes only the mistakes in your original code while keeping its logic.)

Final code:

```
def string_match(text, pattern):
```

```
    n, m = len(text), len(pattern)
```

```
    for i in range(n - m + 1):
```

```
        for j in range(m):
```

```
            if text[i + j] != pattern[j]:
```

```
                break
```

```
            if j == m - 1:
```

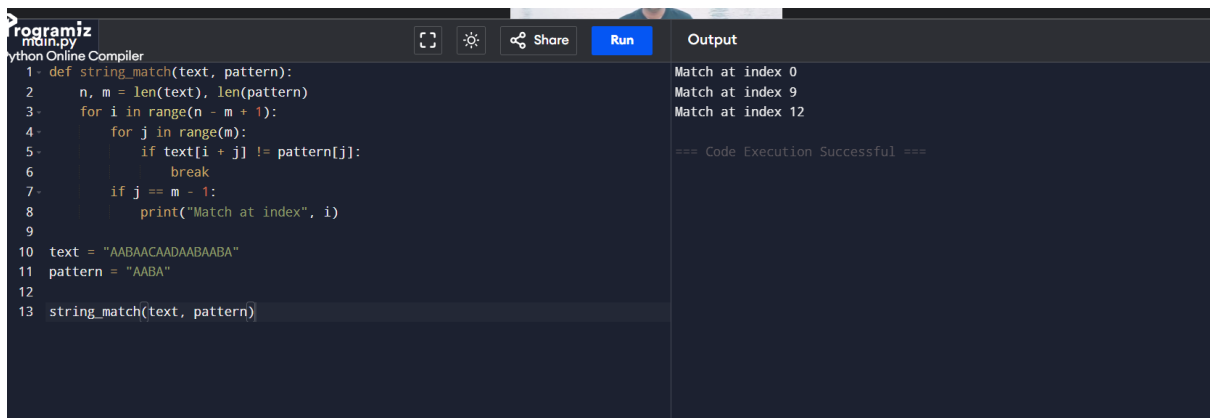
```
                print("Match at index", i)
```

```
text = "AABAACAADAABAABA"
```

```
pattern = "AABA"
```

```
string_match(text, pattern)
```

Output:



The screenshot shows the Programiz Python Online Compiler interface with the final code and its output. The code editor on the left contains the following Python code:

```
1- def string_match(text, pattern):
2-     n, m = len(text), len(pattern)
3-     for i in range(n - m + 1):
4-         for j in range(m):
5-             if text[i + j] != pattern[j]:
6-                 break
7-             if j == m - 1:
8-                 print("Match at index", i)
9-
10- text = "AABAACAADAABAABA"
11- pattern = "AABA"
12-
13- string_match(text, pattern)
```

The output panel on the right displays the result of the code execution:

```
Match at index 0
Match at index 9
Match at index 12

=== Code Execution Successful ===
```

How it works

- It compares the **pattern** with the **text** one character at a time.
- If all characters match, it prints the starting index.
- If a mismatch occurs, it shifts the pattern by one position and tries again.
- This process continues until all possible positions in the text have been checked.

Algorithm

Time Complexity

- **Best Case:** $O(n)$
- **Worst Case:** $O(n \times m)$

where:

- **n** = length of the text
- **m** = length of the pattern.

The given program implements the **Naive (Brute Force) String Matching Algorithm**.